# Witness

A human-in-the-loop decision desk for high-stakes eligibility screening

| | |
|---|---|
| Team | Glue Sticks |
| Event | Agentika 2.0 - NaSCon 2026, FAST NUCES Islamabad |
| Theme | Human + AI: Better Together |
| Live service | https://alternate-echo-whale-5578-github-app.alternatefutures.ai |
| Repository | https://github.com/anithepani/GlueSticks-agentika |
| Submitted | 17 October 2026 |

*One-line pitch:* ***A human-in-the-loop decision desk that screens benefits applications with grounded AI and requires a named officer to decide.***

# 1. Team details

| Role | Name | Email | Discord |
|---|---|---|---|
| Member 1 | Aryan Zia | aryanzia2005@gmail.com | anithepani |
| Member 2 | Muhammad Rubait Zakria | rubait2006@gmail.com | [ADD DISCORD USERNAME] |

Team contact (WhatsApp): **[ADD WHATSAPP NUMBER]**

Team name: **Glue Sticks**

# 2. About the project

**Project name.** Witness

**One-line pitch.** A human-in-the-loop decision desk that screens benefits applications with grounded AI and requires a named officer to decide.

**The problem and who it is for.**

Public benefits screening is high-volume and high-stakes, yet fully automated scoring puts the cost of error on the applicant: a wrong denial has no owner and no reviewable basis. Witness keeps the model advisory, makes every rule deterministic, and makes a named officer accountable for every final decision.

**Where the human comes in.**

The human is a structural stage, not a reviewer bolted on afterwards. The automated pipeline can reach Verified or Needs review; it has no code path that produces a denial. A final decision requires an officer name and a written reason, which are appended to a hash-chained ledger. When evidence is missing, contradictory or uncited, the verifier abstains and escalates the case instead of guessing. The model may only make the outcome more cautious, never less.

**What we built with.**

Python and FastAPI, deployed on Alternate Clouds from a public GitHub repository. Fact extraction uses Groq's llama-3.3-70b-versatile with verbatim-quote grounding; if the model is unavailable it falls back to a deterministic label reader. Policy is evaluated by a deterministic seven-clause rules engine in code. PDF text extraction uses pypdf. The audit ledger is an append-only SHA-256 hash chain in SQLite. No n8n workflow; no MCP servers.

**What the AI may not do**

| Capability | Automated pipeline | Human officer |
|---|---|---|
| Extract facts with citations | Yes | Reviews them |
| Decide a policy clause | No — code does | May override, with a reason |
| Recommend an outcome | Yes | May accept |
| Produce a final denial | Never | Yes, with a written reason |
| Clear a verifier abstention | No | Yes, recorded as an override |

# 3. Required links

| Item | Link |
|---|---|
| Live deployment (Alternate Clouds) | https://alternate-echo-whale-5578-github-app.alternatefutures.ai |
| GitHub repository | https://github.com/anithepani/GlueSticks-agentika |

| Item | Link |
|---|---|
| Demo video (2-3 minutes) | [ADD UNLISTED YOUTUBE OR DRIVE LINK] |
| Original LinkedIn post | [ADD LINKEDIN POST LINK] |
| Teammate reshare | [ADD RESHARE LINK] |
| X / Twitter (optional) | Not posted |

The application has no login: every link above opens directly for the judges. No API keys or personal credentials appear in the repository, the video or these screenshots.

# 4. Proof

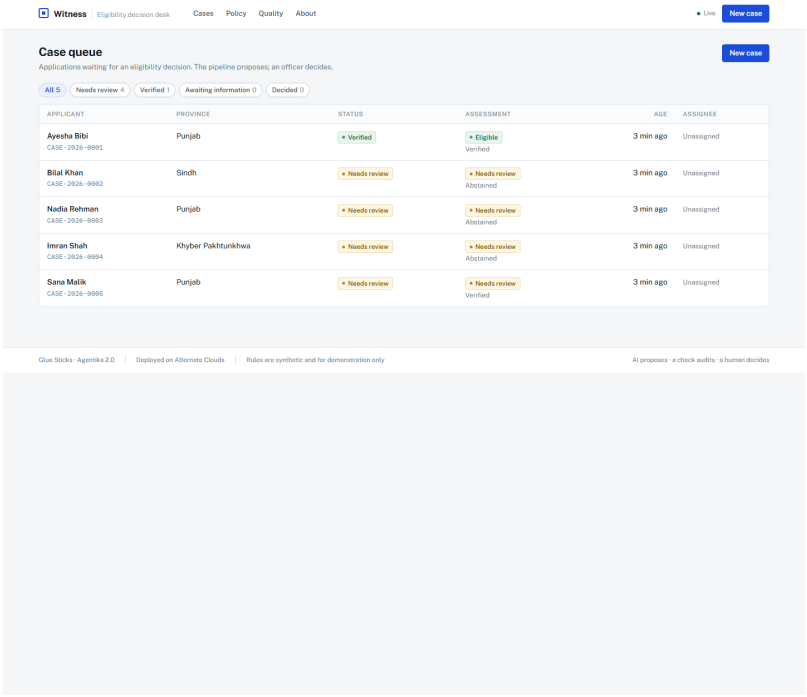
### Discord — both team members in the Alternate Futures server

**[ADD SCREENSHOT: Discord membership, both members]**

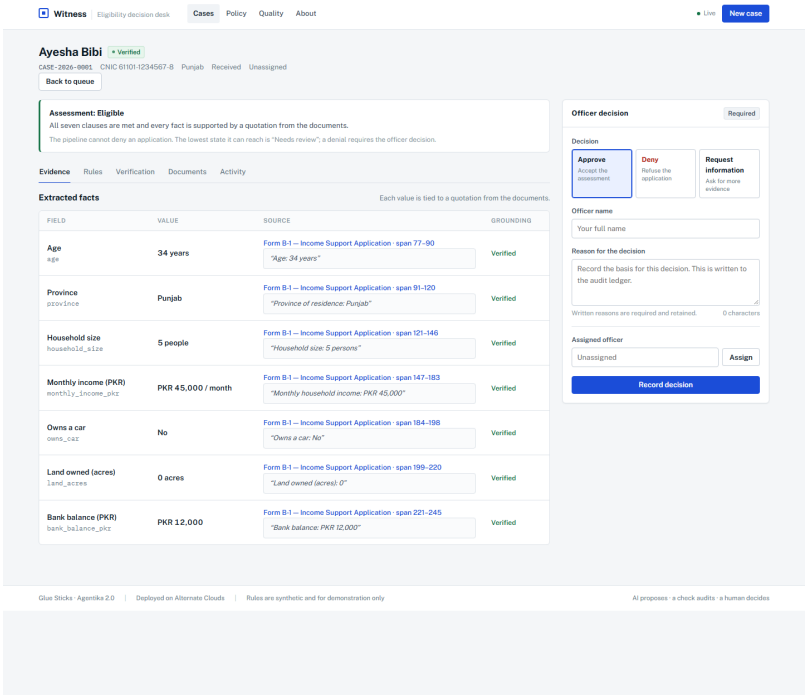
### Alternate Clouds — service and deployment status

**[ADD SCREENSHOT: Alternate Clouds dashboard (do not show environment variables)]**

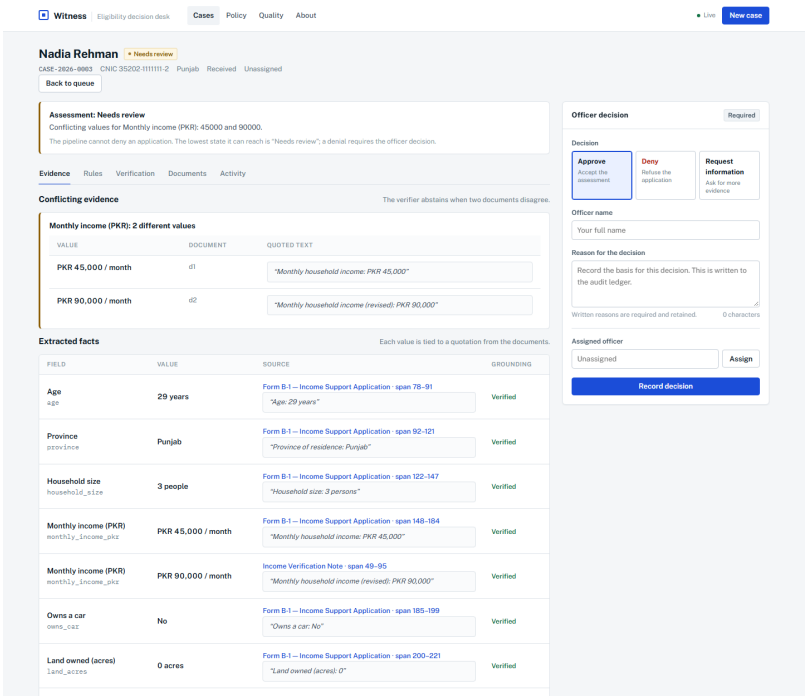
# Application



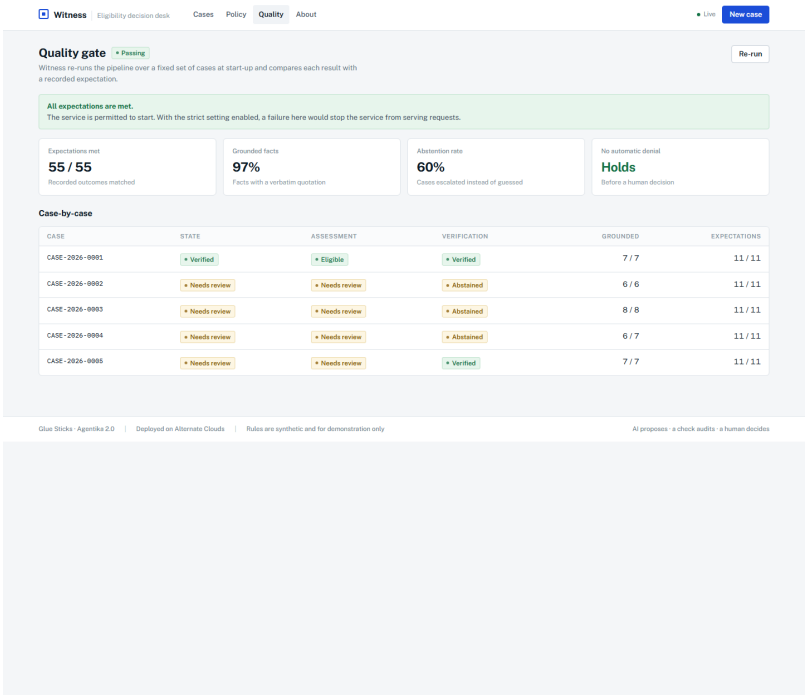

Queue




Evidence




Conflict




Quality

# 5. Confirmations

**Confirmed.** We wrote our LinkedIn post ourselves. Only grammar-only tools were used.

**Confirmed.** Our project is deployed on Alternate Clouds and will stay running until the closing ceremony.

**Confirmed.** No API keys or personal or production passwords appear in our repository, video or screenshots.

**Confirmed.** Third-party components are credited: FastAPI, Uvicorn, pypdf, ReportLab, and a hosted Groq model.

**Confirmed.** We will not change our code or deployment after 12:45 PM PKT on 17 October.

**Confirmed.** The policy we demonstrate is simplified and synthetic; it is not official government policy.

**Verification at submission time.** The live service reports v0.5.0, an evaluation gate of 55/55 expectations with grounding 0.971 and abstention 0.6, and the invariant that no case is automatically denied. The audit chain verifies on the running ledger.

# Where the Human Comes In

A structural human gate for high-stakes AI decisions

| | |
|---|---|
| Project | Witness - a human-in-the-loop decision desk |
| Event | Agentika 2.0 (NaSCon'26, FAST NUCES Islamabad) |
| Theme | Human + AI: Better Together |
| Live service | https://alternate-echo-whale-5578-github-app.alternatefutures.ai |
| Version / build | 0.5.0 |
| Date | 17 October 2026 |

Witness screens applicants for a simplified, synthetic income-support programme. A language model extracts evidence; ordinary code decides the rules; an independent check audits the result; and a human officer makes the decision. The question this document answers is narrow and deliberate: **where, exactly, does the human come in, and what stops the machine from deciding without them?**

> *The short answer: the human is not a reviewer bolted on after the fact. The automated pipeline is structurally incapable of producing a denial. Its strongest possible output is **NEEDS_REVIEW**. **DENIED** exists only as the result of an explicit human action, recorded with the officer's name and a written reason in a tamper-evident ledger.*

## 1. The failure we design against

In benefits screening the costly errors are asymmetric. A wrongly denied applicant may lose food support; a wrongly approved one costs money. Worse, an automated denial is hard to contest because nobody is accountable for it. Two common designs fail here:

• **The autonomous scorer.** A model outputs approve/deny. It is fast, but a denial has no owner and no reviewable basis.

• **The cosmetic reviewer.** A person clicks 'confirm' on whatever the model produced. The human step exists on the diagram but has no authority and no information advantage.

Witness rejects both. The machine is allowed to be useful without being allowed to be final, and the human is given both the authority and the evidence needed to exercise it.

## 2. The rule: the human is a stage in the pipeline

Case state is a closed set. The automated pipeline can only move a case into VERIFIED or NEEDS_REVIEW. Both are pre-decision states. A final state requires a human decision.

| State | Reached by | Final? |
|---|---|---|
| INTAKE | Case received | No |
| VERIFIED | Every rule PASS, every citation grounded, verifier OK | No - awaiting officer |
| NEEDS_REVIEW | A rule FAILed, a rule is UNKNOWN, or the verifier abstained | No - awaiting officer |
| APPROVED | Human officer approves, with a written reason | Yes |
| DENIED | Human officer denies, with a written reason | Yes |
| INFO_REQUESTED | Human officer asks for more evidence | No - case reopens |

There is no code path in which the pipeline writes APPROVED or DENIED. This is enforced by the store's decision gate, not by a prompt instruction: only the human decision endpoint can set a final state, and it refuses to act on a case that is not in a decidable state.

## 3. The pipeline, stage by stage

| Stage | What it does | What it may not do |
|---|---|---|
| Reader | Turns a text or PDF case packet into documents (PDF pages become citable units). | Invent evidence. |
| Extractor | Proposes a value plus the exact quote it came from; the quote is re-grounded verbatim. | Return a fact without a verbatim citation. |
| Rules engine | Ordinary Python compares grounded facts to published thresholds; each rule cites a clause. | Let the model decide a rule. |
| Verifier | Re-audits grounding, contradictions and undecided rules; may only ADD an abstention. | Clear a deterministic failure or approve a case. |
| Human officer | Approves, denies, or requests information - with a mandatory written reason. | Decide without an identity or a reason. |
| Audit ledger | Appends a SHA-256 hash-chained entry for every state change. | Rewrite history undetected. |

Two properties matter more than the individual stages. First, the model's opinion can only make the outcome *more cautious*: the LLM verifier may add an abstention but never removes a rule failure. Second, every claimed fact must survive a verbatim check against its source document, so a hallucinated quote produces no citation and the verifier abstains.

## 4. Where the human acts

An officer sees the evidence - each fact next to the exact quoted span - the rule results, the verifier's checks, and any conflicting values side by side. They then choose one of three actions:

• **Approve** or **deny**: a final decision. A denial is possible *only* here.

• **Request more information**: the case becomes INFO_REQUESTED. New evidence is appended and the pipeline re-runs from the top; the case reopens with no decision attached.

The backend enforces the terms of the act, not just the UI: the officer name must be non-empty, the reason must contain real text, the action must be one of the three, and the case must be in VERIFIED or NEEDS_REVIEW. Duplicate decisions and invalid transitions are rejected. When an officer decides a case the verifier had abstained on, the ledger records **overrode_abstention: true**, so the override is visible rather than silent.

## 5. Abstention instead of guessing

The verifier is designed to refuse. It abstains, and forces NEEDS_REVIEW, when any of the following is true:

• a fact's citation quote does not appear **verbatim** in its source document;

• the same field carries **conflicting** values across documents;

• a rule cannot be decided because its evidence is missing, unreadable, or contradictory.

This is the opposite of a confident chatbot. An abstention is a successful outcome: it routes an uncertain case to the person who is allowed to resolve it, instead of manufacturing a plausible answer. In the live evaluation set, 3 of 5 seeded cases abstain for exactly these reasons.

## 6. Rules are code, not vibes

Seven clauses are evaluated deterministically. The model never decides a rule, and each result names the clause it implements, so a decision traces back to written policy.

| Clause | Rule id | Plain rule | Threshold |
|---|---|---|---|
| 1 | AGE_ADULT | Applicant is 18 or older | age >= 18 |
| 2 | RESIDENCY | Resident in a covered region | province in covered list |
| 3 | HOUSEHOLD_SIZE | Household size recorded | household_size >= 1 |
| 4 | INCOME_LIMIT | Monthly income within limit | monthly_income_pkr <= 60,000 |
| 5 | ASSET_CAR | No car declared | owns_car == false |
| 6 | ASSET_LAND | Land holding within limit | land_acres <= 2 |
| 7 | BANK_BALANCE | Liquid savings within limit | bank_balance_pkr <= 50,000 |

**Disclaimer.** The rule set is simplified, synthetic and BISP-inspired. It is a demonstration and does not represent the official policy of BISP or of any government or agency. Thresholds are illustrative.

## 7. Accountability: a hash-chained ledger

Every state change is appended to an append-only ledger. Each entry stores the hash of the previous entry, so altering or removing a single field breaks verification at that point. **GET /api/audit/verify** recomputes the whole chain and reports the first break.

The ledger is also written **before** in-memory state changes. If the ledger cannot be written, the API returns 503 and the case is left unchanged. A decision can therefore never appear to succeed without a record. This matters because a verifying chain proves *integrity*, not *durability*; Witness is explicit about the difference.

A live demonstration endpoint, **POST /api/audit/tamper-demo**, tampers with an in-memory copy of the ledger, shows the chain breaking exactly at the forged entry, and then re-verifies the untouched ledger to show the real record is intact.

## 8. What the AI is not allowed to do

| Capability | Automated pipeline | Human officer |
|---|---|---|
| Extract facts with citations | Yes | Reviews them |
| Decide a policy rule | No (code does) | May override with a reason |
| Produce ELIGIBLE recommendation | Yes | May accept |
| Produce a final denial | **Never** | Yes, with a written reason |
| Clear a verifier abstention | No | Yes, recorded as overrode_abstention |
| Write the audit entry | Yes | The entry names the officer |

## 9. Evidence from the live service

| Check | Live result (17 Oct 2026) |
|---|---|
| Startup evaluation gate | 55 / 55 expectations met; all_passed = true |
| Grounding rate | 0.971 (facts carrying a verbatim citation) |
| Abstain rate | 0.6 (cases escalated instead of guessed) |
| No-auto-deny invariant | true (pre-human states: INTAKE, VERIFIED, NEEDS_REVIEW) |
| Audit chain | verifies on the live ledger |
| Deployed build | v0.5.0 |

## 10. Honest limits

• **The ledger is local to the deployment.** It survives a process restart and is reset when the service is redeployed. A verifying hash chain proves integrity, not durability, and the interface states that rather than implying otherwise.

• **The policy is synthetic.** The rules are a demonstration of structure, not real eligibility law. Thresholds are illustrative.

• **Unfamiliar documents abstain.** If a pasted case does not contain the expected form labels, the fields stay unknown and the case is escalated instead of guessed.

## 11. Verify it in sixty seconds

Open https://alternate-echo-whale-5578-github-app.alternatefutures.ai and try the following. Each step is a live check on the production service.

| What to do | What it proves |
|---|---|
| Open /api/eval | The gate re-runs: 55/55, grounding, abstain rate, no_auto_deny. |
| Open a case with a contradiction (CASE-2026-0003) | The verifier abstains; state is NEEDS_REVIEW, never DENIED. |
| Try to deny with an empty reason | The backend returns 400; a human act requires a reason. |
| Approve a case, then reopen its audit trail | The officer, reason, timestamp and hash are recorded. |
| Open the Activity tab and press 'Verify ledger' | The chain recomputes to an intact head. |
| Press 'Simulate tampering' | A forged copy breaks at the changed entry; the real ledger stays intact. |
| Open About | The page states exactly which reader, model, verifier and ledger are running. |

> *The design claim is deliberately modest and testable: the machine may propose, an independent check may refuse, and only a named person may decide. Everything else in Witness exists to make that sentence true in code, visible in the interface, and provable from the ledger.*

Live service: https://alternate-echo-whale-5578-github-app.alternatefutures.ai

Repository: https://github.com/anithepani/GlueSticks-agentika